

```

#include <SPI.h>
#include <Ethernet.h>
#include <SoftwareSerial.h>
#include <PZEM004Tv30.h>

// --- CONFIGURATION RÉSEAU ---
byte mac[] = { 0xDE, 0xAD, 0xBE, 0xEF, 0xFE, 0xED };
IPAddress ip(172, 29, 254, 50);          // IP de ton Arduino
IPAddress dnsServer(172, 29, 254, 1);    // Serveur DNS
IPAddress gateway(172, 29, 254, 1);      // Passerelle (Gateway)
IPAddress subnet(255, 255, 255, 0);      // Masque de sous-réseau

IPAddress server(172, 29, 254, 14);      // IP du serveur de l'Étudiant 4
const int port = 3000;                  //
EthernetClient client;                  //

// --- CONFIGURATION CAPTEURS ---
SoftwareSerial pzemSWSerial(2, 3);       // RX sur 2, TX sur 3
PZEM004Tv30 pzem(pzemSWSerial);         //

// --- VARIABLES DE SESSION ---
bool sessionEnCours = false;            //
unsigned long debutChrono = 0;           //
unsigned long dureeTotale = 0;           //

void setup() {
    Serial.begin(115200);                //

    // CORRECTION : Déclaration propre des paramètres réseau pour éviter
    // l'erreur de compilation
    Ethernet.begin(mac, ip, dnsServer, gateway, subnet);

    delay(1000);                         //
    Serial.print("Système prêt ! IP : ");
    Serial.println(Ethernet.localIP());
}

// Fonction pour vérifier l'état de la session sur le serveur
bool verifierSessionServeur() {
    if (client.connect(server, port)) { //
        client.println("GET /api/session-status HTTP/1.1"); // Route à
        // créer par l'étudiant 4
        client.println("Host: 172.29.254.14"); //
    }
}

```

```

client.println("Connection: close");           //
client.println();                             //

// Lecture de la réponse
while (client.connected()) {
    if (client.available()) {
        String line = client.readStringUntil('\r');
        if (line.indexOf("\"active\":true") != -1) {
            client.stop();
            return true;
        }
        if (line.indexOf("\"active\":false") != -1) {
            client.stop();
            return false;
        }
    }
}
client.stop();
}
return false;
}

// Fonction pour envoyer la durée finale une fois la session terminée
void envoyerDureeSession(unsigned long duree) {
    if (client.connect(server, port)) {
        String json = "{\"duree_session\": " + String(duree) + "}";
        client.println("POST /api/fin-session HTTP/1.1");
        client.println("Host: 172.29.254.14:3000");
        client.println("Content-Type: application/json");
        client.print("Content-Length: ");
        client.println(json.length());
        client.println();
        client.println(json);
        client.stop();
        Serial.println("Session terminee. Duree envoyee : " + String(duree)
+ "s");
    }
}

void loop() {
    // 1. Vérification de l'état de la session (Polling)
    bool etatActuel = verifierSessionServeur();

```

```

// Détection du DÉBUT de session
if (etatActuel == true && sessionEnCours == false) {
    sessionEnCours = true;
    debutChrono = millis();
    Serial.println("Début de session détecté !");
}

// Détection de la FIN de session
if (etatActuel == false && sessionEnCours == true) {
    sessionEnCours = false;
    dureeTotale = (millis() - debutChrono) / 1000; // Conversion en
secondes
    envoyerDureeSession(dureeTotale);
}

// 2. Lecture des données capteurs
float v_pzem = pzem.voltage();
float p_pzem = pzem.power();
int raw_acs = analogRead(A0);
int raw_rs = analogRead(A1);

// 3. Connexion et envoi classique des données capteurs
if (client.connect(server, port)) {
    // Construction du message JSON
    String json = "{";
    json += "\"v_pzem\": " + String(isnan(v_pzem) ? 0 : v_pzem) + ",";
    json += "\"p_pzem\": " + String(isnan(p_pzem) ? 0 : p_pzem) + ",";
    json += "\"acs_0\": " + String(raw_acs) + ",";
    json += "\"rs_solaire\": " + String(raw_rs) + ",";
    json += "\"session_active\": " + String(sessionEnCours ? "true" :
"false");
    json += "}";

    // Envoi de la requête HTTP POST sur la bonne route
    client.println("POST /api/consos HTTP/1.1");
    client.println("Host: 172.29.254.14:3000");
    client.println("Content-Type: application/json");
    client.println("Connection: close");
    client.print("Content-Length: ");
    client.println(json.length());
    client.println(); // Ligne vide cruciale
    client.println(json); // Corps du message

```

```
    Serial.println("Données envoyées : " + json);
    client.stop();
} else {
    Serial.println("Erreur : Impossible d'envoyer les données
capteurs");
}

delay(5000); // Envoi toutes les 5 secondes
}
```